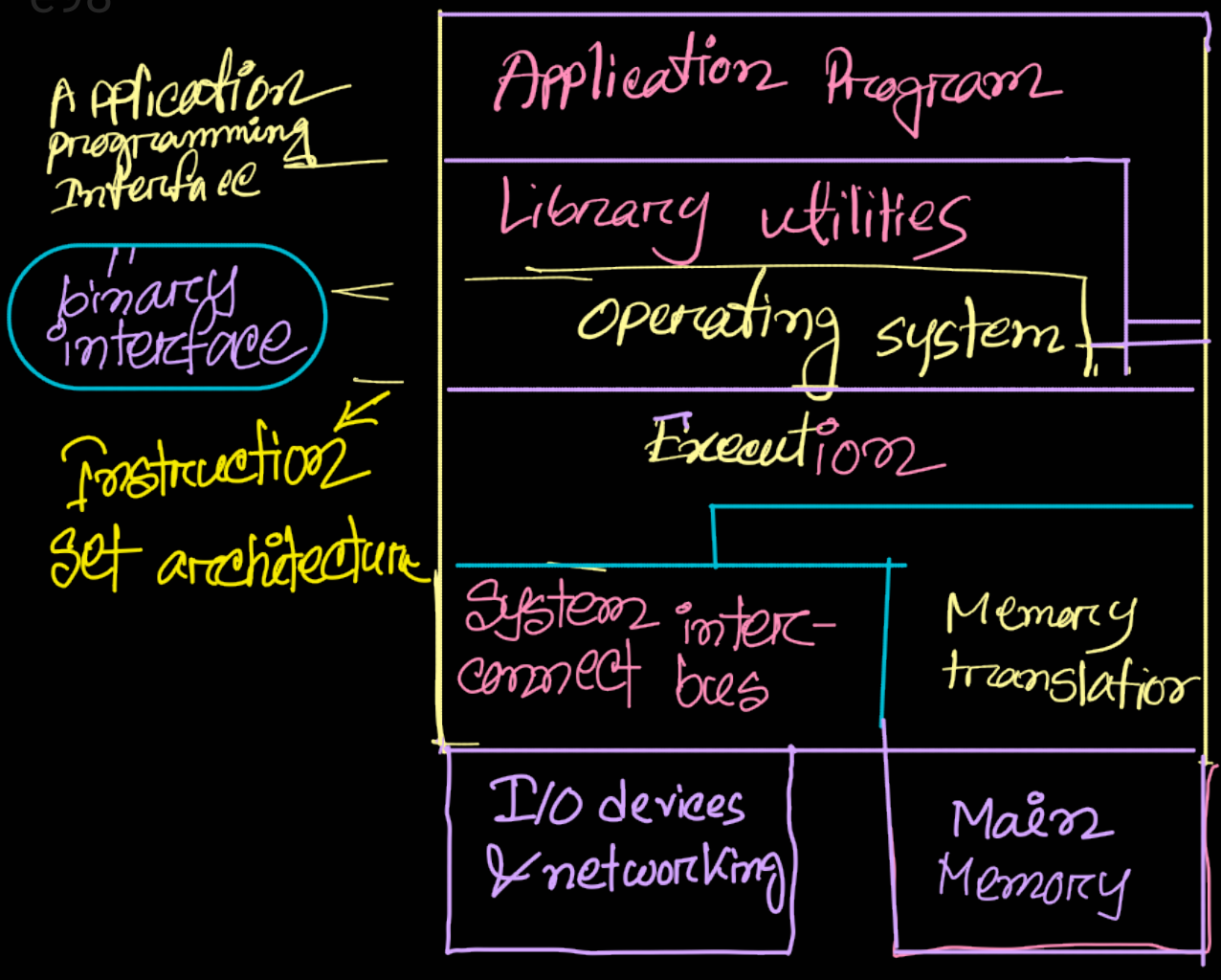
e98



Utilities → (System programs)

implement frequently used functions:
(help in)

↳ program creation

↳ file management

↳ I/O device control.

programmers apps — use facilities → develop apps

apps — invoke utilities to → perform functions.

Most important system program

↳ OS.

↳ 2 objectives:

(1) making comp more convenient to use.

(2) allowing comp system resources to be used efficiently.

OS provides service in:

(not part of OS) utility prog ↑

(1) Prog creation: using facilities & services,
: editors & debuggers.

(2) Prog execution:

a) load data & instn to → M.M.

b) Initialize → I/O device & files.

(3) Access to I/O device:

specific set of instruction/control signal

for all I/O device

(4) Controlled access to files :

understand → nature of I/O device
→ file format on storage medium.

(+ protection mechanism with multiple users)

(5) System access :

(i) control access to system as a whole
+ specific resources.

(ii) protect from unauthorized users &
resolve conflicts for resource contention.

(6) Error Detection & response :

internal & external hardware errors.

↳ memory error
↳ Device failure / malfunction.

Software errors :

↳ arithmetic overflow
↳ out of bound memory access.

Inability of OS to grant req to an app.

Soln: OS takes action to resolve error with least impact on applications.

(i) end prog
(ii) retry the operation
(iii) reporting the error to the app.

(7) Accounting: A good OS,

→ collects usage statistics for various resources.

→ monitors performance parameters (response time)

→ anticipate future enhancement need,

↓

on multiuser system used for billing purpose.

3 Key Interfaces in comp system:

→ Architecture

① ISA : ML instr. comp can follow.

↑ → (h/w & s/w boundary) ✳

App prog & utilities directly access it.

use ISA: a subset of ISA for them

☆ System ISA: Deal with system management resource

(Additional ML instr)

↳ OS has access.

② Application Binary Interface: ABI

A standard binary portability across systems.

A system call interface → to → OS

→ to → h/w resources & services.

③ Application programming Interface: API

gives

access to h/w resource & services

↓ through

allows User ISA + HLL library calls. (does sys call)

App s/w to be ported easily

↓ through

recompilation

to other sys with same API

Resource Management

Computers — A set of resources for ↓

movement, storage & processing of data & control of these func.

↳ OS manages these. But not entirely.

↓

1. Functions like a normal comp s/w.

2. Frequently releases control & depends on processor to give it again.

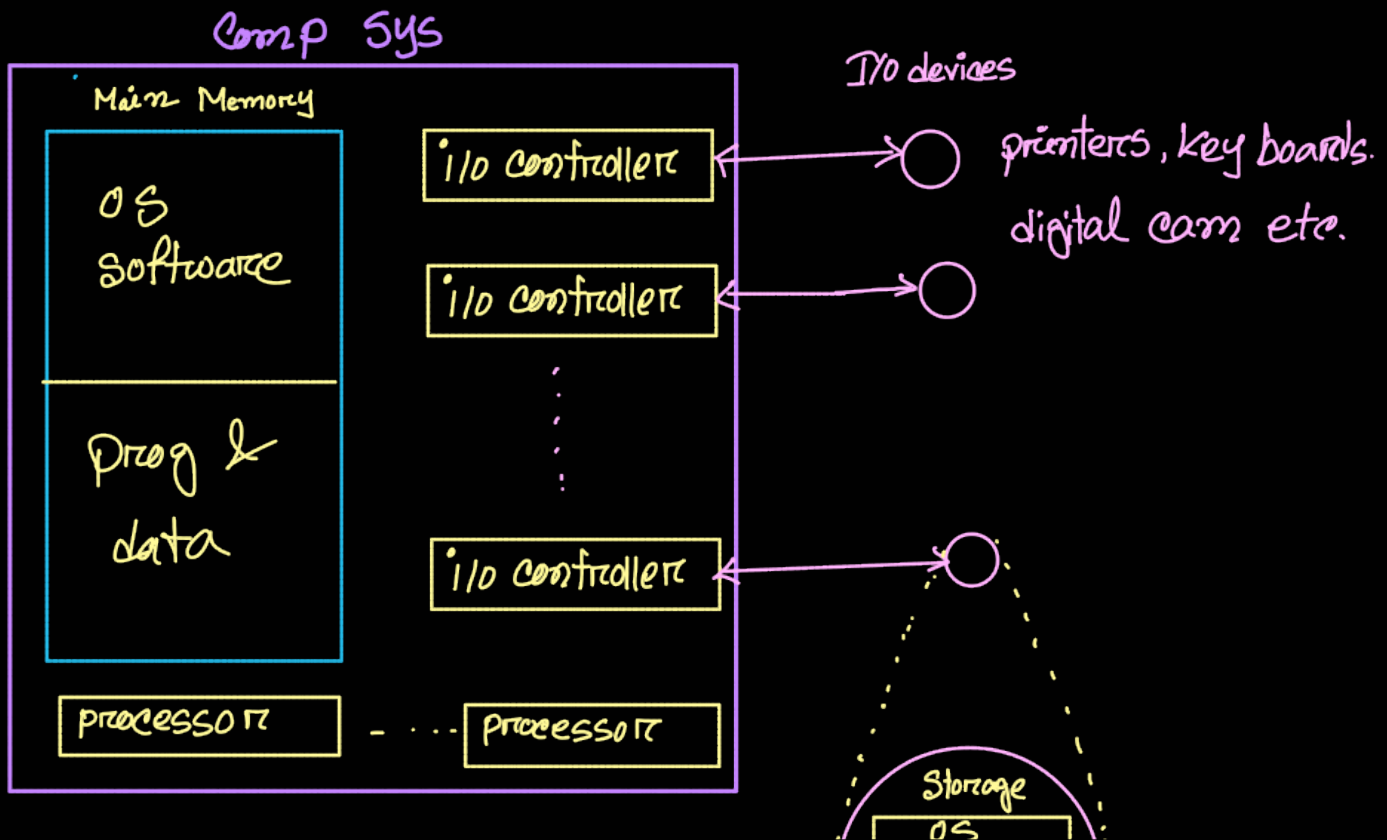
OS provide instr. for processor.

↳ Directs in use of sys resources.

↳ timing of execution of progs.

So proc stops executing OS prog.

So OS releases control & takes back long enough to prepare it to next piece of work.





OS

↓

A portion in MM

↳ Kernel

↓ has

Freq used func

↳ other portions of OS currently being used

The allocation of M.M is jointly controlled by

→ OS &

→ Memory-management h/w in processor.

World 2. **Types of OS**

↓

batch/interactive

↓

User's prog batched with other user prog & submitted by comp operator.

↓

directly interact with comp

Multi / uni programming

↓

Keep proc as busy as possible.

Several prog loaded onto mem & proc switches rapidly among them

Early systems

programmer directly interacted with h/w without OS. used consoles with display lights, toggle switches, input device (card reader), printers. Programs manually loaded & identify errors through indicator lights.

2 main probms

1. Scheduling: users reserved processor time in fixed blocks ⟹ wasted idle time or incomplete tasks.
2. Setup: Multiple manual steps to run a prog:
   - → loading compilers
   - → linking
   - → Mounting tapes or card decks.

Error Occurs! ⟶ restart (Inna lillah)

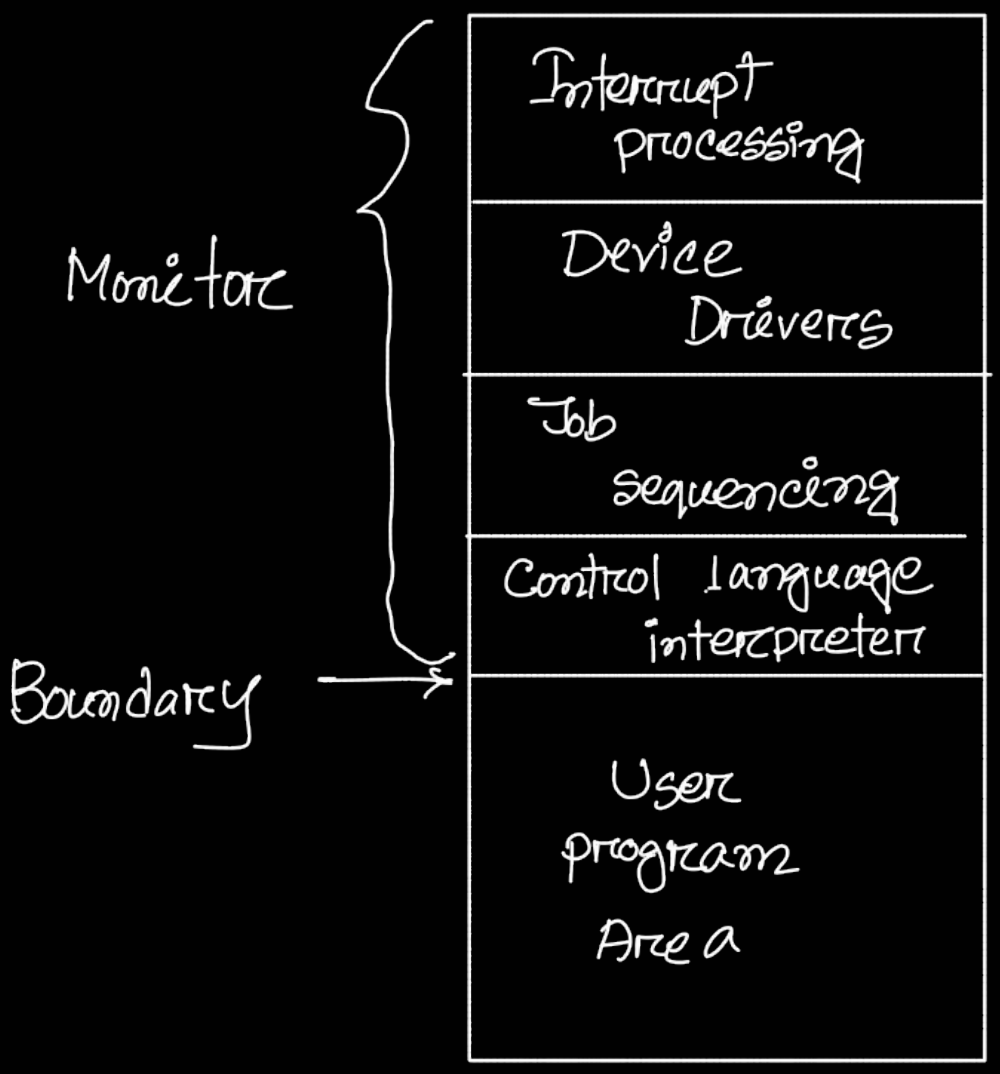
↳ Serial processing

grants user exclusive access to comp

in sequence.

To improve ⇒ library, loaders, linkers, debuggers, I/O drivers were introduced.

Simple Batch Processing



Early expensive processors so to reduce idle time & maximize proc utilization.

Basic operation:

Users don't interact directly with proc. they submit jobs (via punched cards/ mg tape) to a comp operator → batches them together for sequential processing.

The Role of Monitor

☐ Resident Monitor:

⇒ A core part of the batch OS kept in M.M at all times.

⇒ reads jobs from input device, loading them onto the user prog area, transferring control to user prog.

☐ Job execution cycle:

submit • Job submission by user

Load & execute • The monitor reads a job into memory, transfers control to it &

processor executes the user program.

Return of control

- Once a job is finished, control returns to the monitor → proceeds to load next job.

☐ Job scheduling cycle.

Monitor maintains a queue (batch) of jobs & ensures the processor is never idle by quickly switching between them.

Job control & Setup

Job control lang (JCL): special instr. starting with $ sign. tells the monitor how to process the job including loading proper compiler or loaders.

$FTN: Load Fortran compiler

Compilation: Compiler translates source code to object code.

$LOAD & $RUN: Instruct the monitor to load & run the compiled code.

$END: Job End!

Control transfer:

Proc alternates between executing monitor instro. & user prog instr.

When user prog encounters error or completes execution, proc fetches next instro from monitor.

The Monitor or batch OS is just a computer prog who relies on proc to fetch instro. from M.M to seize & release control alternately.

The hardware features: (supporting Monitor)

① Memory Protection: Ensures that user prog can't alter the monitor's memory area. Any attempt is caught by the proc & results in job termination.

② Timers: Prevents any one job from monopolizing the CPU by setting a maximum execution time.

timer expired ⟶ interrupt returns control to monitor.

③ Privileged instr: Certain instrs (I/O operations specially) can only be executed by the monitor. If a user prog attempts to use these, an error interrupt is triggered.

④ Interrupts: Early models didn't have it, interrupts allowed the monitor to regain control more flexibly from the user progs.

Advantages:

1. Maximizes proc utilization by reducing idle time.
2. Allows large segment of memory among different subsystems, Even tho only 1 is active at a time.

Overheads

⇒ A portion of M.M always reserved for res. Monitor.

⇒ Some proc time is consumed by the monitor's management task (e.g. job scheduling, mem prote).

Multi Programmed batch system

*Limitations of Uni programming:

In a simple (uni) ", the processor often

remains idle while waiting for slow I/O devices to complete their task.

* Multi prog/ tasking :

⊙ With additional M.M, the OS can hold the OS + 2 or more progs at once.

Benefit: While one job is waiting for I/O, another can use the proc → overall CPU utilization. ↑

⊙ Hardware Support: ☆

1. I/O interrupts: Allow proc to switch tasks as soon as an I/O operation completes.

2. DMA (Direct Mem Acc): Enables data transfer between devices & mem without continuous CPU intervention.

⊙ OS Responsibilities:

1. Mem Management: Multiple jobs must

reside in M.M. requiring effective memory partitioning.

2. Job scheduling: The OS uses scheduling algos to decide which job to run next.

Time-Sharing Systems

multiple users simultaneously access system thru terminal.

- **Need for Interactivity:**

As computing evolved, many applications required direct user interaction (transaction processing).

- **How Time Sharing Works:**

Interleaved Execution: The OS rapidly switches between multiple user progs by assigning short bursts (time slices) of CPU time.

User experience:

= For n users, each gets roughly $\frac{1}{n}$ of the proc's capacity.

comparable to dedicated comps.

- Despite sharing, the response time remains quick enough for interactive tasks cz human reaction times are slower than CPU speeds

! Ha egpt roasts.

- **Directive sources**:

→ Batch Multiprog: Uses JCL commands provided with the job.

→ Time sharing: Commands are entered interactively at the terminal.

	Batch Mult	Time share
Principle objective	Maximize proc use.	Minimize response time
Source of Directive	JCL command	terminal command

↓ increases CPU use by overlapping

↓ Slices proc time

i/o & processing